

Agentic FacilityOps AI Platform

Energy Intelligence & Monitoring

Technical Project Documentation

Version 1.0 | Energy Intelligence Module

Date: 05 August 2026

Author: G Vinay Kumar | Agentic AI Engineering

Table of Contents

1. Project Overview & Objectives	pg. 3
2. Problem Statement & Business Impact	pg. 3
3. Technology Stack	pg. 4
4. System Architecture	pg. 5
5. Database Design	pg. 6
6. Backend — FastAPI Application	pg. 7
7. AI Energy Agent — Rule Engine	pg. 8
8. Analytics Engine	pg. 9
9. REST API Endpoints	pg. 10
10. Frontend — React Dashboard	pg. 11
11. Dashboard Pages & Features	pg. 12
12. Data Simulation & CSV Layer	pg. 14
13. Report Export Engine	pg. 14
14. Licenses & Open Source	pg. 15
15. How to Run the Application	pg. 15
16. Future Roadmap	pg. 16

1. Project Overview & Objectives

Agentic FacilityOps AI Platform is a production-grade, enterprise-level AI-powered Energy Intelligence and Monitoring system for commercial buildings such as IT Parks, Hospitals, Universities, Shopping Malls, and Factories. The platform collects energy consumption data from IoT sensors (simulated via CSV files), analyses energy usage using Python data science tools, detects abnormal consumption through an AI rule engine, generates intelligent recommendations, and visualises everything through a real-time professional dashboard similar to Microsoft Azure Monitor, AWS CloudWatch, or Grafana.

1.1 Core Objectives

Objective	Description
Real-Time Monitoring	Collect and display hourly electricity, water, HVAC, solar, and power factor sensor readings.
AI Anomaly Detection	Automatically detect energy spikes, HVAC overloads, power factor drops, night anomalies, and water leaks.
Intelligent Recommendations	Generate prioritised energy-saving action plans with estimated kWh and cost savings.
Analytics & KPIs	Compute carbon footprint, efficiency score, peak demand hours, and facility-wise comparisons.
Report Export	Export audit-ready reports in CSV, Excel (.xlsx), and PDF formats.
Scalable Architecture	Designed to extend to Maintenance, Occupancy, Security, and Cost Intelligence modules.

2. Problem Statement & Business Impact

Commercial buildings consume **40–60% of all electricity produced globally**. Most facility managers have no real-time visibility into energy patterns, no automated alerts for anomalies, and no AI-driven guidance for reduction — relying only on monthly billing reports.

Challenge	Traditional Approach	Our Solution
Real-time visibility	Monthly utility bills	Live IoT telemetry dashboard
Anomaly detection	Manual inspection (delayed)	AI Agent auto-detects within the hour
Energy optimisation	Expensive consultant reports	Automated AI recommendations with savings
Multi-facility comparison	Manual spreadsheets	Automated benchmark comparison matrix
Compliance reporting	Manual data collection	One-click CSV / Excel / PDF export

Business Impact: AI-driven energy monitoring can reduce electricity bills by 10-25% per facility, equivalent to Rs.5-20 Lakh savings annually per large commercial building.

3. Technology Stack

3.1 Backend

Technology	Version	License	Purpose
Python	3.11+	PSF (Free)	Core programming language
FastAPI	>=0.110	MIT	REST API web framework with auto Swagger docs
SQLAlchemy ORM	>=2.0	MIT	Database ORM — write Python instead of SQL
Pydantic	>=2.6	MIT	Automatic request/response data validation
Pandas	>=2.2	BSD	Vectorised analytics and CSV data processing
NumPy	>=1.26	BSD	Numerical computations for baseline calculations
ReportLab	>=4.1	BSD	PDF audit report generation
OpenPyXL	>=3.1	MIT	Excel (.xlsx) report generation
Uvicorn	>=0.28	BSD	ASGI server for FastAPI

3.2 Frontend

Technology	Version	License	Purpose
React	v19	MIT	Component-based UI framework
Vite	v6.4	MIT	Ultra-fast dev server and production build tool
Tailwind CSS	v4	MIT	Utility-first CSS — all styling via class names
Chart.js	v4.4	MIT	Canvas-based interactive charting engine
React-ChartJS-2	v5.3	MIT	React wrapper for Chart.js components
React Router	v7	MIT	Client-side SPA routing (no page reloads)
Axios	v1.7	MIT	HTTP client for API communication
Lucide React	v0.475	ISC	Professional SVG icon library

3.3 Database

Database	Use Case	License
SQLite	Local development — zero config, file-based (facilityops.db)	Public Domain
PostgreSQL	Production — enterprise RDBMS via DATABASE_URL env variable	PostgreSQL License

4. System Architecture

The platform follows a clean **three-tier architecture**: Presentation Layer (React frontend), Application/Business Logic Layer (FastAPI + AI Agent + Analytics Engine), and Data Layer (SQLite/PostgreSQL). The layers communicate strictly through the REST API contract.

```
PRESENTATION LAYER (React 19 + Vite + Tailwind CSS)
Dashboard | Energy Monitoring | Analytics | Alerts | Recommendations | Reports | Settings
| HTTP REST (Axios)
APPLICATION LAYER (FastAPI Python)
API Routers: /facilities /energy /analytics /alerts /recommendations /reports
| |
AI Energy Agent Analytics Service
Rule Engine (5 rules) Pandas + NumPy (15+ KPI metrics)
Recommendations Engine Carbon footprint / Efficiency score
| SQLAlchemy ORM
DATA LAYER (SQLite / PostgreSQL)
Tables: facilities | energy_usage | energy_alerts
^
IoT DATA SOURCE LAYER
CSV Simulation (current) --> MQTT/WebSocket IoT Sensors (future)
```

5. Database Design

Table 1: facilities

Column	Type	Constraint	Description
facility_id	VARCHAR(50)	PRIMARY KEY	Unique ID (e.g., FAC-001)
facility_name	VARCHAR(150)	NOT NULL	Full building name
facility_type	VARCHAR(100)	NOT NULL	IT Park / Hospital / University / Shopping Mall / Factory
city	VARCHAR(100)	NOT NULL	City location
state	VARCHAR(100)	NOT NULL	State/Province
total_floors	INTEGER	DEFAULT 1	Number of floors
total_area_sqft	FLOAT	DEFAULT 10000	Total area in sq.ft

Table 2: energy_usage (18,000+ IoT records)

Column	Type	Description
energy_id	INTEGER (PK)	Auto-incrementing primary key
facility_id	VARCHAR(50) FK	Foreign key → facilities.facility_id
timestamp	DATETIME	Hourly IoT sensor reading timestamp
electricity_kwh	FLOAT	Total electricity consumed (kWh)
water_liters	FLOAT	Water consumption (litres/hr)
hvac_kwh	FLOAT	HVAC system electricity usage (kWh)
lighting_kwh	FLOAT	Lighting electricity usage (kWh)
solar_generation_kwh	FLOAT	Rooftop PV solar generation (kWh)
power_factor	FLOAT	Electrical power factor (cos phi, 0-1)
temperature	FLOAT	Ambient temperature (degrees C)
humidity	FLOAT	Relative humidity (%)

Table 3: energy_alerts (AI-Generated)

Column	Type	Description
alert_id	INTEGER (PK)	Auto-incrementing primary key
facility_id	VARCHAR(50) FK	Foreign key → facilities.facility_id
timestamp	DATETIME	When the anomaly was detected
severity	VARCHAR(20)	Critical High Medium Low
alert_type	VARCHAR(100)	High Energy Consumption / HVAC Overload / Power Spike / Water Leakage / Abnormal Night Usage

Column	Type	Description
message	TEXT	Data-driven incident description
recommendation	TEXT	AI-generated action recommendation text
status	VARCHAR(20)	Open In Progress Resolved

6. Backend — FastAPI Application

The backend follows **Clean Architecture** principles — separated into Models, Schemas, Agents, Services, and API layers. On startup it auto-creates tables and seeds 18,000+ records.

Startup Sequence

- **Step 1:** FastAPI instantiated with CORS middleware.
- **Step 2:** SQLAlchemy creates all DB tables if they don't exist.
- **Step 3:** data_service.py generates CSV files (30 days × 5 facilities = 18,000+ hourly records).
- **Step 4:** Database seeded from CSV if tables are empty.
- **Step 5:** All API routers mounted under /api prefix.
- **Step 6:** Uvicorn serves on http://0.0.0.0:8000.

Folder Structure

```
backend/  
app/  
main.py FastAPI entry point, startup lifecycle  
core/  
config.py Pydantic Settings (env vars, thresholds)  
database.py SQLAlchemy engine and session factory  
models/  
facility.py Facility ORM model  
energy.py EnergyUsage ORM model  
alert.py EnergyAlert ORM model  
schemas/ Pydantic request/response validation schemas  
api/  
facilities.py GET /api/facilities  
energy.py GET + POST /api/energy  
analytics.py GET /api/analytics  
alerts.py GET + PATCH /api/alerts  
recommendations.py GET /api/recommendations  
reports.py GET /api/reports/export  
agents/  
energy_agent.py AI rule engine + recommendations generator  
services/  
analytics_service.py Pandas analytics engine (15+ metrics)  
data_service.py CSV generator and DB seeder  
data/  
sample_facilities.csv  
sample_energy_data.csv
```

7. AI Energy Agent — Rule Engine

The **AI Energy Agent** (`energy_agent.py`) is the intelligence core. It runs automatically when new telemetry is ingested (`POST /api/energy`) or when alerts are fetched. It uses a statistical baseline approach combined with rule-based anomaly detection.

7.1 How the Agent Works (Step by Step)

- 1. Loads last 720 hours (30 days) of hourly records for each facility from the database.
- 2. Converts to Pandas DataFrame, sorts chronologically, extracts hour-of-day column.
- 3. Groups by hour-of-day to compute the 7-day hourly baseline (statistical mean).
- 4. Retrieves the most recent record as the subject for evaluation.
- 5. Applies 5 detection rules sequentially.
- 6. Deduplication: same alert type per facility won't re-fire within 12 hours.
- 7. Saves new alerts to `energy_alerts` table and commits the session.

7.2 Detection Rules

Rule #	Rule Name	Condition	Severity	Alert Type
Rule 1	Energy Spike	Electricity > 20% above 7-day hourly baseline	High / Critical (>40%)	High Energy Consumption
Rule 2	HVAC Overload	HVAC kWh > 50% of total facility electricity	High	HVAC Overload
Rule 3	Power Factor Drop	Power Factor < 0.90 (utility penalty threshold)	Medium	Power Spike
Rule 4	Night Anomaly	Off-hours (23:00-05:00) consumption > 65% of daytime avg	Medium	Abnormal Night Usage
Rule 5	Water Leakage	Night water flow > 1.5x facility average	Critical	Water Leakage

7.3 AI Recommendations Engine

Analyses 7-day patterns to generate prioritised energy-saving recommendations. Each includes: Priority, Est. Cost Savings (USD/month), Est. Energy Savings (kWh), Root Cause, Suggested Action.

Recommendation Title	Trigger	Category	Priority
Optimize HVAC Thermostat & Reset Setpoint	HVAC > 46% of total load	HVAC	High
Automate After-Hours Lighting Shutdown	Night lighting high	Lighting	Medium
Install APFC Capacitor Banks	Avg Power Factor < 0.92	Power Factor	High/Medium
Solar PV Panel Cleaning & Inspection	Solar generation present	Solar	Low

8. Analytics Engine

The analytics service uses **Pandas vectorised operations** to compute 15+ metrics from database records. Powers all KPI cards, charts, and tables on the dashboard.

Metric	Calculation	Used In
Total Electricity (kWh)	df['electricity_kwh'].sum()	KPI Card
Today vs Yesterday %	((today-yesterday)/yesterday)*100	KPI trend arrow
Average Hourly kWh	df['electricity_kwh'].mean()	KPI sub-text
Peak Consumption Hour	hourly_grp.idxmax() on groupby hour	Analytics heatmap
Energy Efficiency Score	95 - PF penalty - HVAC ratio penalty (0-100 scale)	KPI & comparison table
Carbon Footprint (Tons CO2e)	total_kWh x 0.82 kg/kWh / 1000	Analytics & KPI card
Hourly Trend (24h)	Last 24 hours individual records	Line chart
Daily Trend (7 days)	groupby(date).sum().tail(7)	Bar chart
Monthly Trend (30 days)	groupby(date).sum().tail(30)	Area chart
Category Breakdown %	HVAC% + Lighting% + Equipment% + Solar%	Doughnut chart
Facility Comparison	groupby(facility_id) aggregate all metrics	Bar chart & matrix table
24-Hour Peak Heatmap	Mean kWh per hour 00:00-23:00	Analytics heatmap grid
Active & Critical Alert Count	COUNT WHERE status != Resolved	KPI Card

Efficiency Score Formula: $\text{Score} = \max(50, \min(98, 95 - \max(0, (0.95 - \text{avg_pf}) \times 100) - \max(0, (\text{hvac_ratio} - 0.45) \times 50)))$ | **Grid Emission Factor:** 0.82 kg CO2/kWh

9. REST API Endpoints

All endpoints at <http://localhost:8000>. Swagger UI auto-generated at <http://localhost:8000/docs>.

Method	Endpoint	Description	Query Params
GET	/api/facilities	List all commercial facilities	—
GET	/api/energy	Fetch energy telemetry records	facility_id, limit
GET	/api/energy/{facility_id}	Fetch records for one facility	limit
POST	/api/energy	Ingest new IoT reading; triggers AI Agent	JSON body
GET	/api/analytics	Full analytics computation (15+ metrics)	facility_id, start_date, end_date
GET	/api/alerts	Get AI-generated energy alerts	facility_id, severity, status
PATCH	/api/alerts/{id}/status	Update alert status	Body: {status}
GET	/api/recommendations	Get prioritised AI recommendations	facility_id
GET	/api/reports/export	Download audit report file	format, facility_id

10. Frontend — React Dashboard

Single-page application built with **React 19 + Vite + Tailwind CSS**. Follows a component-driven architecture. Design inspired by Microsoft Azure Portal — dark glassmorphism cards, top navbar with global facility selector, fixed left sidebar.

Reusable UI Components

Component	File	Purpose
	layout/Navbar.jsx	Logo, facility dropdown, alert badge, theme toggle
	layout/Sidebar.jsx	7 nav items, active highlighting, AI Engine status panel
	common/GlassCard.jsx	Glassmorphism container with title, subtitle, action slot
	common/KpiCard.jsx	Metric card with value, unit, trend %, icon, sub-text
	charts/HourlyLineChart.jsx	24h multi-line telemetry chart (Chart.js Line)
	charts/WeeklyBarChart.jsx	7-day grouped bar chart (Chart.js Bar)
	charts/CategoryPieChart.jsx	End-use category doughnut chart
	charts/FacilityBarChart.jsx	Facility comparison bar chart
	charts/MonthlyAreaChart.jsx	30-day area trend + solar overlay
	common/IngestDataModal.jsx	Modal form to POST new IoT readings

Global State — React Context API

FacilityContext provides: selectedFacilityId (updates all pages when changed), theme (dark/light), facilities list (dropdown), refreshTrigger (forces data re-fetch), isIngestModalOpen.

11. Dashboard Pages & Features

/ Dashboard

Executive overview. 5 KPI cards (Today Electricity, Water, HVAC, Active Alerts, Efficiency Score). 5 charts: 24h Line, 7-day Bar, Category Doughnut, Facility Bar, 30-day Area. Active Alerts table (top 5). AI Recommendations list (top 3). Data loaded via 4 parallel Axios calls.

/energy Energy Monitoring

Real-time IoT telemetry control room. 4 live gauges: Power Factor (colour-coded), HVAC Load %, Solar Generation kWh, Temperature/Humidity. Searchable raw telemetry table (150 records). 'Ingest Sensor Metric' button opens POST form.

/analytics Analytics

Deep intelligence. Carbon footprint card (Tons CO2e). 24-hour peak demand heatmap grid. ASHRAE-style 0-100 efficiency rating. Full facility performance comparison matrix table.

/alerts Alerts

AI incident management. Filter by Severity (Critical/High/Medium/Low) and Status (Open/In Progress/Resolved). Each card shows alert details + AI recommendation. Inline Acknowledge & Investigate / Mark Resolved buttons.

/recommendations Recommendations

Prioritised AI action plan cards. Aggregate savings banner (total \$/month + kWh/month). Each card: Priority, Category, Est. Cost Savings, Est. Energy Savings, Root Cause, Suggested Action, 'Implement' button.

/reports Reports

Compliance export. One-click CSV / Excel (.xlsx) / PDF download. Configurable facility scope and report type selector (Full Log / Anomalies Only / Carbon Audit).

/settings Settings

Configure AI Agent rules (Spike %, HVAC %, Power Factor threshold). Dark/Light theme toggle. Platform roadmap panel showing upcoming modules.

12. Data Simulation & CSV Layer

Synthetic telemetry generated with **NumPy random seed 42** for reproducibility. The CSV layer is a swappable service — replacing CSV with live MQTT/WebSocket IoT streams requires changes only in `data_service.py` with zero frontend modifications.

Facility	Type	City	Base Load (kWh/hr)
CyberTech IT Park	IT Park	San Jose, CA	320
St. Jude General Hospital	Hospital	Chicago, IL	480
Pacific Innovation University	University	Seattle, WA	400
Grand Horizon Shopping Mall	Shopping Mall	Dallas, TX	350
Apex Precision Factory	Factory	Detroit, MI	550

Total: 5 facilities x 30 days x 24 hours = 3,600 records/facility = 18,000+ total hourly IoT records

Realistic Patterns Simulated

- Diurnal business curve — higher load 08:00-19:00, lower at night.
- Weekend reductions — IT Parks and Factories run at 50% load on weekends.
- Solar generation — follows a sine curve from 07:00 to 18:00.
- 3% deliberate anomaly injection — random spikes for AI Agent detection testing.
- HVAC overload events — 50% chance of HVAC spike when electricity anomaly fires.
- Low power factor events — 4% probability per record.

13. Report Export Engine

Format	Library Used	Content
CSV	Python csv module	Full hourly IoT sensor telemetry stream (all 11 columns)
Excel (.xlsx)	OpenPyXL >= 3.1	Formatted workbook with column headers, data rows, auto-sizing
PDF	ReportLab >= 4.1	Formal audit document: cover, summary stats, compliance headers, sample telemetry

14. Licenses & Open Source

All tools, frameworks, and libraries are 100% free and open-source (MIT / BSD / PostgreSQL licenses). No paid licenses, API keys, or cloud subscriptions are required. The platform can be used for personal, academic, and commercial purposes without any licensing fees.

15. How to Run the Application

Prerequisites

- Python 3.11+
- Node.js 18+ and npm
- Git (optional)

Start Backend (FastAPI)

```
cd backend
python -m uvicorn app.main:app --host 0.0.0.0 --port 8000 --reload
```

Backend auto-creates tables and seeds data. Swagger UI: <http://localhost:8000/docs>

Start Frontend (React + Vite)

```
cd frontend
npm run dev
```

Dashboard available at: <http://localhost:5173>

Key Environment Variables

Variable	Default	Purpose
DATABASE_URL	sqlite:///./facilityops.db	Switch to PostgreSQL for production
ELEVATED_SPIKE_PERCENT	20.0	Energy spike detection threshold (%)
HVAC_ALERT_PERCENT	50.0	HVAC overload alert threshold (%)
MIN_POWER_FACTOR	0.90	Power factor minimum before alert fires
CORS_ORIGINS	['http://localhost:5173']	Allowed frontend origins

16. Future Roadmap

Module	Description	Status
Energy Intelligence & Monitoring	Real-time IoT telemetry, AI anomaly detection, recommendations, export reports	ACTIVE
Predictive Maintenance	ML-based equipment failure prediction (HVAC, lifts, pumps) using vibration + runtime data	Upcoming
Occupancy & Space Optimization	People-count analytics, zone utilisation, desk/conference room efficiency scoring	Upcoming
Security & Anomaly Intelligence	Access control monitoring, CCTV event classification, perimeter breach alerts	Upcoming
Cost Intelligence & Billing	Tariff analysis, demand charge optimisation, ROI calculator, carbon credit tracking	Upcoming

End of Documentation — Agentic FacilityOps AI Platform v1.0 | Energy Intelligence & Monitoring

Document generated: 05 August 2026, 04:45 PM